

APACHE SPARK VS DATABRICKS

 **Core Truth (Memorize This)**

Spark = Engine (technology)

Databricks = Platform (environment to use Spark easily)

1. What is Apache Spark?

 **Definition:**

An **open-source distributed processing engine** used to process large-scale data.

 **What it does:**

- Reads data
- Processes data
- Writes data

 **Where it runs:**

- On clusters (Hadoop, Kubernetes, standalone)
-

 **Spark Capabilities:**

- Batch processing
 - Streaming
 - SQL (Spark SQL)
 - Machine Learning (MLlib)
-

 **But here's the problem:**

Using Spark alone means:

- You manage clusters
- You handle failures
- You configure memory, nodes
- You manage scaling

 That's **hard in real production**

2. What is Databricks?

Definition:

A **cloud-based data platform** built by the creators of Spark.

What it provides:

- Ready-to-use Spark environment
 - No cluster headaches
 - Built-in tools for data engineering, ML, analytics
-

In simple words:

Databricks = **Spark + Cloud + Tools + Automation**

3. Architecture Difference

Using Spark Alone

You manage everything:

- Cluster setup
- Resource tuning
- Job scheduling
- Monitoring

 More control, but more work

Using Databricks

Databricks handles:

- Cluster creation
- Auto scaling
- Optimization
- Monitoring

 You focus on **data logic only**

4. Key Differences (Crystal Clear Table)

Feature	Spark	Databricks
Type	Engine	Platform
Setup	Manual	Fully managed
Ease of use	Medium/Hard	Easy
Scaling	Manual	Auto scaling
UI	Limited	Rich notebooks & dashboards
Collaboration	No	Yes
Optimization	Manual tuning	Built-in optimizations
Cost	Infra cost only	Platform + infra cost

5. Databricks Extra Power (This is WHY companies use it)

◆ 1. Delta Lake (Game changer)

- ACID transactions on data lakes
 - Handles:
 - Updates
 - Deletes
 - Versioning
-

◆ 2. Collaborative Notebooks

- Like Jupyter but better
 - Teams work together
-

◆ 3. Auto Optimization

- Query optimization
 - Caching
 - File compaction
-

◆ 4. Integrated Ecosystem

Works directly with:

- AWS S3
 - Azure Data Lake
 - Google Cloud Storage
-

6. Real-World Analogy

Think like this:

Concept Example

Spark Engine of a car

Databricks Full car (engine + dashboard + automation)

👉 You *can* build a car from an engine

👉 But Databricks gives you the **complete ready car**

⚠️ 7. Important Reality (Most beginners miss this)

Databricks **USES Spark internally**

So:

- When you learn Spark → You are learning Databricks core
 - When you use Databricks → You are still running Spark jobs
-

🔥 8. When to Use What?

Use Spark directly if:

- You want deep control
 - Working on low-level optimization
 - Learning fundamentals
-

Use Databricks if:

- Production systems
 - Fast development needed
 - Team collaboration
 - Cloud-based pipelines
-

9. Interview Answer (Perfect)

If asked:

“Difference between Spark and Databricks?”

Say:

“Apache Spark is an open-source distributed processing engine, while Databricks is a cloud-based platform built on top of Spark that provides managed infrastructure, collaborative tools, and optimizations, making it easier to develop and run big data workloads.”

Apache Spark vs Databricks

Apache Spark

- Open-source distributed data processing engine
- Designed to process **large-scale data across clusters**
- Can run **on-premise or in the cloud**

Key idea:

Spark is just the **engine** — you manage setup and execution.

Databricks

- A **cloud platform built on top of Apache Spark**
- Provides a **fully managed Spark environment**

Simple analogy:

- Spark = Engine
 - Databricks = Car (engine + automation + comfort)
-

What Databricks Adds

- Managed clusters (no manual setup)
- Optimized Spark performance
- Cloud integration (AWS, Azure, GCP)
- **Delta Lake** for reliable storage

- Collaboration via notebooks
- Security & governance

👉 **Why companies prefer it:**

- Faster development
- Less operational work
- Better performance

🧠 **Spark Architecture**

📦 **Layer 1: Low-Level (Spark Core / RDD)**

- Works with **RDDs**
- Languages: Python, Scala, Java, R
- High control, but complex

📦 **Layer 2: High-Level APIs (Most Used)**

- DataFrames
- Spark SQL
- Structured Streaming
- MLlib
- GraphX

👉 **Used by ~90% of data engineers**

⚖️ **RDD vs DataFrame vs Spark SQL**

Feature	RDD	DataFrame	Spark SQL
Difficulty	Hard	Medium	Easy
Flexibility	High	Medium	Low
Performance	Low	High	Very High
Optimization	Manual	Automatic	Automatic

🌱 **RDD (Resilient Distributed Dataset)**

- Distributed data collection

- Fault-tolerant
- Immutable

! **Limitations:**

- No optimization
 - Manual control required
-

 DataFrame

- Structured data (rows & columns)
- Faster than RDD
- Uses **Catalyst optimizer**

👉 Best balance of performance + usability

 Spark SQL

- Query data using SQL

```
SELECT * FROM users WHERE age > 25;
```

👉 Best for:

- Analysts
 - Simple transformations
-

 Core Data Processing Flow

● **1. LOAD (Ingestion)**

- Sources: HDFS, S3, databases, APIs

```
df = spark.read.csv("data.csv")
```

● **2. TRANSFORM**

- filter, join, groupBy, aggregation

```
df.filter(df.age > 25)
```

● **3. WRITE**

- Store results in warehouse, lake, or files

```
df.write.parquet("output/")
```

Pipeline Summary

Raw Data → Transform → Store Output

RDD Internals

RDD = Resilient Distributed Dataset

- **Resilient** → Recovers from failures
- **Distributed** → Spread across nodes
- **Dataset** → Collection of data

 Internally:

- Data split into **partitions**
 - Processed in **parallel**
 - Failures handled via **lineage (recomputation)**
-

Spark Core APIs (RDD Level)

- Direct control over transformations
- No automatic optimization

```
rdd.map().filter().reduce()
```

High-Level APIs (Industry Standard)

DataFrames

- Structured, fast, easy

Spark SQL

- SQL-based processing

Structured Streaming

- Real-time processing

MLlib

- Machine learning

GraphX

- Graph processing
-

Key Insight

Spark = Engine

RDD = Low-level control

DataFrame = Balanced approach

Spark SQL = Simplest usage

Databricks = Managed Spark platform

Real-World Example

E-commerce (Flipkart / Amazon)

1. Load

- User clicks, orders, logs

2. Transform

- Filter valid users
- Aggregate sales
- Join product data

3. Write

- Store in data warehouse
-

Final Interview Summary

- Spark is a **distributed processing engine**
- Databricks is **managed Spark on cloud**
- RDD = powerful but complex
- DataFrames & SQL = faster and easier
- Every Spark job follows:

Load → Transform → Write

HOW SPARK ACTUALLY WORKS INTERNALLY DURING EXECUTION?

🔥 Spark Internals (What REALLY happens under the hood)

Most people stop at:

“Spark is fast, distributed, DataFrames are optimized”

That’s not enough for interviews or real-world debugging.

You need to understand:

Code → Logical Plan → Optimized Plan → Physical Plan → Execution

💡 1. Step-by-Step Spark Execution Flow

Let’s take a simple example:

```
df.filter(df.age > 25).groupBy("city").count()
```

What YOU see:

Simple transformation

What SPARK does internally:

🟢 Step 1: Logical Plan (Unoptimized)

Spark first creates a **logical plan (tree structure)**

Example:

GroupBy(city)

↑

Filter(age > 25)

↑

Scan(data source)

👉 No execution yet

👉 Just planning

🟡 Step 2: Catalyst Optimizer (Brain of Spark)

This is where magic happens.

🔥 Catalyst does:

- Predicate pushdown (filter early)
- Column pruning (only needed columns)
- Constant folding
- Join reordering

👉 Example optimization:

Instead of:

Load full data → Filter later

Spark does:

Filter while reading data

👉 Huge performance gain

🔵 Step 3: Physical Plan

Now Spark decides:

👉 *HOW exactly to execute*

Example decisions:

- Hash aggregation vs Sort aggregation
 - Broadcast join vs Shuffle join
-

🔴 Step 4: DAG Creation

Spark converts plan → **DAG (Directed Acyclic Graph)**

👉 This is the execution workflow

Stage 1 → Stage 2 → Stage 3

⚙️ Step 5: Task Execution

- Data split into partitions
 - Each partition = 1 task
 - Tasks run in parallel across cluster
-

⚡ 2. What makes Spark FAST?

Let's be very clear — this is where Spark beats Hadoop.

🔥 1. In-Memory Processing

- Stores intermediate data in RAM
- Avoids disk reads/writes

👉 Compare:

System Storage

Hadoop Disk ❌

Spark Memory ✅

🔥 2. Lazy Evaluation

Spark does NOT execute immediately.

👉 It waits until an **action** is called

Example:

```
df.filter().groupBy()
```

❌ Not executed

```
df.show()
```

✅ Now execution starts

🔥 3. DAG Optimization

Instead of step-by-step execution (like Hadoop):

Spark combines operations into **optimized stages**

📦 3. Transformations vs Actions (CRITICAL)

🟡 Transformations (Lazy)

- filter
- map
- groupBy

👉 Only builds plan

🔴 Actions (Trigger execution)

- `show()`
- `count()`
- `collect()`
- `write()`

👉 Starts actual computation

⚠️ 4. Shuffle (BIGGEST PERFORMANCE KILLER)

This is where many jobs fail in real world.

! What is shuffle?

👉 Data moves across machines

Example:

```
df.groupBy("city")
```

All same city data must go to same node

👉 Network transfer = slow

🚨 Problems caused:

- High network I/O
 - Disk spill
 - Slow jobs
-

✅ How to reduce shuffle:

- Use **partitioning**
 - Use **broadcast joins**
 - Avoid unnecessary `groupBy`
-

🧠 5. Partitioning (Hidden Superpower)

👉 Spark processes **partitions in parallel**

Key rule:

More partitions → More parallelism → Faster execution

But:

Too many → Overhead

Too few → Underutilization

👉 Balance is key (INTERVIEW FAVORITE)

🔥 6. Why DataFrames are Faster than RDD

This is where many people give weak answers. Don't.

! Real reasons:

✅ 1. Catalyst Optimizer

RDD → No optimization

DataFrame → Fully optimized

✅ 2. Tungsten Engine

- Better memory management
 - Uses binary format
 - CPU efficient
-

✅ 3. Code Generation

Spark generates **optimized JVM bytecode**

👉 Faster than generic execution

🔪 7. Spark vs Hadoop (REAL Difference)

Feature	Hadoop	Spark
---------	--------	-------

Execution	Step-by-step	DAG
-----------	--------------	-----

Speed	Slow	Fast
-------	------	------

Storage	Disk	Memory
---------	------	--------

Flexibility	Low	High
-------------	-----	------
